

Fondamentaux de PowerShell, Système d'Aide et Modèle Objet .NET

1.1 La rupture avec les interpréteurs textuels traditionnels

Les interpréteurs de commandes historiques (tels que cmd.exe sous Windows ou les shells Unix traditionnels comme sh ou bash) font transiter des flux de caractères non structurés (*raw text streams*) via les descripteurs standards stdout et stdin. Dans ces environnements, l'interopérabilité entre deux utilitaires impose un traitement textuel complexe (découpage par colonnes, expressions régulières via sed, awk ou cut), hautement sensible aux variations de formatage ou de localisation linguistique de l'interface système.

PowerShell repose sur un paradigme radicalement distinct :

- **Le transit d'objets en mémoire vive** : Les commandes n'émettent pas du texte, mais des instances d'objets fortement typées issues du runtime .NET (*Common Language Runtime - CLR*).
- **Conservation de la sémantique** : Une commande transmet un objet complet contenant des attributs typés (entiers, booléens, dates, structures complexes) et des méthodes exécutables. Le filtrage ou le tri ne requiert aucune analyse syntaxique de chaînes de caractères.
- **Découplage strict entre données et affichage** : La présentation visuelle en console (mise en forme sous forme de table ou de liste) est une opération terminale gérée par le sous-système de formatage de PowerShell (Format-Table, Format-List), indépendante de la donnée sous-jacente.

1.2 La nomenclature canonique : Verbe-Nom

Afin d'uniformiser l'administration du système d'exploitation et des services distants, PowerShell impose une règle de nommage stricte pour ses commandes natives (nommées *cmdlets*) :

CMDLET = Verbe + Noun

- **Le Verbe (Verb)** : Identifie l'action précise exécutée par la commande. Microsoft maintient une liste fermée de verbes approuvés garantissant la cohérence sémantique globale. Les verbes respectent des catégories fonctionnelles formelles :
 - Cycle de vie : New, Remove, Start, Stop, Restart.
 - Consultation : Get, Find, Search.
 - Modification d'état : Set, Update, Reset.
 - Sécurité et diagnostic : Test, Assert, Protect.

- **Le Nom (Noun)** : Identifie la ressource technique ciblée par l'action. Il est conventionnellement rédigé au singulier (ex. : Process, Service, NetAdapter, EventLog).

La commande Get-Verb permet d'obtenir à tout moment le catalogue officiel des verbes autorisés et leur finalité opérationnelle :

Get-Verb

2. Découverte et Cartographie des Commandes (Get-Command)

Face à la multiplicité des modules d'administration système (Active Directory, réseau, stockage, hyperviseurs), l'ingénieur système ne mémorise pas l'intégralité des cmdlets. L'autonomie opérationnelle repose sur la maîtrise de l'annuaire interne via la cmdlet **Get-Command**.

2.1 Fonctionnement et paramètres d'exploration

Get-Command interroge la table des commandes enregistrées dans la session et les modules déclarés dans les répertoires d'environnement (\$env:PSModulePath).

Filtrage par motifs textuels génériques (*Wildcards*)

L'usage de l'astérisque (*) permet d'isoler des commandes par ciblage thématique :

Recherche de toutes les commandes relatives aux services système

Get-Command *Service*

Recherche ciblée sur les composants d'adaptateurs réseau

Get-Command *NetAdapter*

Filtrage par composante syntaxique (-Verb et -Noun)

Lorsque l'action recherchée est identifiée, l'isolation par verbe restreint drastiquement le bruit d'affichage :

Recenser toutes les commandes exécutant une réinitialisation / redémarrage

Get-Command -Verb Restart

Identifier précisément la commande de redémarrage d'une carte réseau

Get-Command -Verb Restart -Noun *NetAdapter*

Résultat obtenu : La commande exacte est **Restart-NetAdapter**.

Recherche par domaine fonctionnel et calcul statistique

Pour mesurer des volumétries, quantifier des lignes ou réaliser des opérations arithmétiques sur des objets sans recourir à des boucles impératives :

Identification des utilitaires de calcul et métrologie

Get-Command *Measure*

Résultat obtenu : La commande centrale de métrologie et d'agrégation statistique est **Measure-Object**.

2.2 Typologie des éléments découverts par Get-Command

La propriété CommandType permet de distinguer la nature technique de l'exécutable sous-jacent :

CommandType	Description technique
Cmdlet	Commande native compilée en langage .NET (C#) intégrée à un module.
Function	Code scripté rédigé en syntaxe PowerShell résidant en mémoire ou dans un module script (.psm1).
Alias	Raccourci pointant vers une cmdlet, une fonction ou un exécutable (ex. : dir → Get-ChildItem).
Application	Binaire exécutable externe situé sur le disque (.exe, .com).

3. Le Système d'Aide Intégré et Analyse Syntaxique (Get-Help)

PowerShell embarque un moteur de documentation exhaustif accessible hors-ligne via la cmdlet **Get-Help**.

3.1 Initialisation et niveaux d'affichage

Sous Windows PowerShell 5.1 et PowerShell 7, l'aide n'est pas préinstallée dans son intégralité afin de limiter l'empreinte disque. La synchronisation initiale s'effectue via :

Update-Help -Force -ErrorAction SilentlyContinue

Granularité de lecture

La cmdlet Get-Help expose plusieurs commutateurs adaptant la verbosité de la documentation :

1. Affichage standard (Description sommaire et syntaxe)

Get-Help Get-Process

2. Affichage exhaustif (Détail de chaque paramètre et contraintes d'entrées/sorties)

Get-Help Get-Process -Full

3. Affichage direct des cas d'usage réels

Get-Help Get-Process -Examples

4. Déport de l'aide dans une interface graphique avec champ de recherche dynamique

Get-Help Get-EventLog -ShowWindow

3.2 Déchiffrement de la syntaxe officielle

L'aptitude la plus critique dans l'usage de PowerShell consiste à interpréter sans ambiguïté les blocs de syntaxe retournés par Get-Help.

Considérons l'extrait de syntaxe officiel de la cmdlet **Stop-Process** :

SYNTAX

Stop-Process [-Id] <System.Int32[]> [-PassThru] [-Force] [-WhatIf] [-Confirm]
[<CommonParameters>]

Stop-Process -Name <System.String[]> [-PassThru] [-Force] [-WhatIf] [-Confirm]
[<CommonParameters>]

Stop-Process [-InputObject] <System.Diagnostics.Process[]> [-PassThru] [-Force] [-WhatIf]
[-Confirm] [<CommonParameters>]

Règle 1 : Les jeux de paramètres mutuellement exclusifs (*Parameter Sets*)

Chaque paragraphe distinct représente un scénario d'exécution exclusif. Il est techniquement impossible de mixer les paramètres uniques de deux jeux distincts au sein du même appel. Pour arrêter un processus, l'opérateur doit choisir d'identifier la cible :

- Soit par son PID numérique (-Id),
- Soit par son libellé exécutable (-Name),

- Soit par un objet processus complet transitant par variable ou pipeline (-InputObject).

Règle 2 : La convention des crochets d'obligation

La disposition typographique des crochets [et] régit le caractère obligatoire ou facultatif des éléments :

- **Paramètre Obligatoire** : Le nom du paramètre et son argument **ne sont pas** encapsulés dans des crochets globaux.
Dans le jeu 2 : -Name <System.String[]>
→ Le paramètre -Name est **strictement obligatoire**. L'omission de cet argument bloque l'exécution et force PowerShell à ouvrir une invite de saisie interactive.
- **Paramètre Optionnel** : L'ensemble formé par le nom du paramètre et son type est encapsulé dans des crochets.
Exemple : [-Force] ou [-PassThru]
→ Ce sont des commutateurs facultatifs modifiant le comportement interne de la commande.

Règle 3 : Les paramètres positionnels

Lorsqu'un paramètre est requis, mais que son nom textuel est lui-même entouré de crochets, le paramètre est qualifié de **positionnel** :

- Dans le jeu 1 : [-Id] <System.Int32[]>
 - Les crochets autour de [-Id] indiquent que la mention explicite du mot -Id est facultative.
 - En revanche, l'argument <System.Int32[]> est obligatoire.
 - Par conséquent, l'instruction Stop-Process 4128 est syntaxiquement valide :
PowerShell assigne implicitement la valeur entière ⁴¹²⁸ au paramètre -Id en vertu de sa position ⁰.
- Dans le jeu 2 : -Name <System.String[]>
 - Le mot -Name n'est pas entre crochets : il est obligatoire de taper le nom du paramètre si la position n'est pas supportée dans ce jeu. L'appel doit obligatoirement s'écrire : Stop-Process -Name "notepad".

Règle 4 : La cardinalité des types (Scalaire vs Tableau)

La présence des caractères [] après le type .NET indique que la cmdlet accepte une collection d'éléments séparés par des virgules :

- <System.String[]> : Accepte un tableau de chaînes (ex. : -Name "notepad", "calc", "msedge").
- <System.Int32> sans crochets : N'accepterait qu'un entier unique.

4. Le Modèle Objet et l'Exploration Réflexive

(Get-Member)

4.1 Le Pipeline orienté objet

Dans les shells traditionnels, le tube (*pipe* |) sérialise la sortie d'un processus en un tableau d'octets ASCII ou UTF-8. L'outil récepteur doit réanalyser la structure du texte :

```
# Approche textuelle classique (Linux/Unix) : obligation de parser les colonnes  
ps aux | grep nginx | awk '{print $2}'
```

En PowerShell, l'opérateur | transfère **la référence mémoire vivante de l'objet .NET** d'une commande à l'autre :

```
# Approche orientée objet (PowerShell) : accès direct aux propriétés membres  
Get-Process -Name notepad | Stop-Process
```

L'objet émis par Get-Process traverse le pipeline avec l'ensemble de ses attributs d'état et ses méthodes d'action intacts.

4.2 L'inspection réflexive avec Get-Member

Pour analyser le contrat d'interface et la composition d'une donnée transitant dans la console, on redirige la commande vers **Get-Member** (alias conventionnel : gm).

```
Get-Date | Get-Member
```

La ligne d'en-tête : TypeName

La première ligne retournée par Get-Member est fondamentale : elle spécifie la classe .NET exacte modélisée par l'objet :

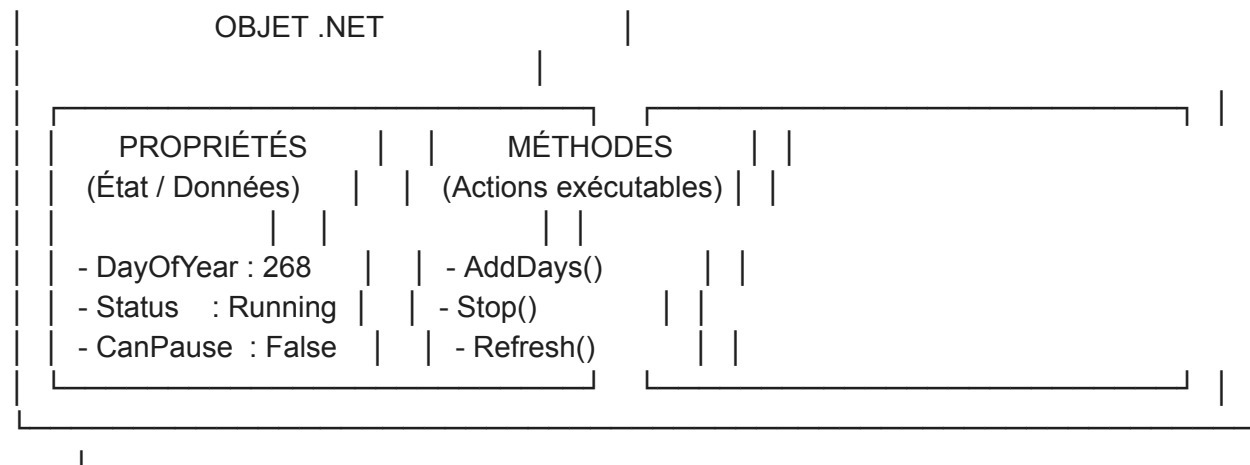
```
TypeName: System.DateTime
```

Cette information détermine l'ensemble des capacités fonctionnelles de la donnée. L'administrateur sait immédiatement qu'il manipule une structure standard Microsoft .NET System.DateTime, et non une simple représentation textuelle d'horloge.

4.3 Typologie des membres : Propriétés vs Méthodes

Un objet .NET combine deux entités cardinales :





1. Les Propriétés (*Properties*)

Elles décrivent l'état passif de l'objet à l'instant t de son instanciation.

- Exemple sur Get-Date : La propriété **DayOfYear** retourne un entier scalaire compris dans l'intervalle 1.366 correspondant au quantième de l'année.

Accès programmatique par notation pointée :

(Get-Date).DayOfYear

2. Les Méthodes (*Methods*)

Elles constituent des fonctions actives embarquées directement dans l'objet, capables de manipuler son état interne ou d'interagir avec le système d'exploitation hôte. Une méthode impose l'usage des parenthèses d'invocation (), recevant le cas échéant des paramètres typés.

- Exemple sur Get-Date : La méthode **AddDays(double value)** calcule et retourne une nouvelle instance temporelle incrémentée ou décrétementée d'une grandeur arithmétique :

Calcul de la date exacte à J+7

(Get-Date).AddDays(7)

Calcul de la date exacte à J-30

(Get-Date).AddDays(-30)

5. Étude de Cas Pratique : Métrologie et Contrôle de Services

L'analyse du service système Windows Update (wuauserv) permet d'observer la puissance opérationnelle de l'inspection d'objets.

5.1 Inspection du composant

Get-Service -Name wuauserv | Get-Member

Restitution de l'inspection :

- **Type .NET retourné :**

Cet objet est un gestionnaire d'infrastructure directement connecté au gestionnaire de contrôle des services Windows (*Service Control Manager* - SCM).

- **Propriété d'interrogation d'aptitude :**

La propriété booléenne **CanPauseAndContinue** indique si le sous-système du service implémente la capacité de mise en pause sans destruction du thread de traitement :

(Get-Service -Name wuauserv).CanPauseAndContinue

Retourne \$false : le service wuauserv ne supporte que l'arrêt complet ou l'exécution.

- **Méthode d'action directe :**

L'objet expose la méthode **Stop()**, permettant de déclencher l'arrêt du service directement au niveau applicatif, sans instancier de cmdlet secondaire :

Exécution de l'action via la méthode de l'objet

\$Service = Get-Service -Name wuauserv

\$Service.Stop()

6. Synthèse des Commandes Piliers

Le tableau suivant récapitule la triade fondamentale garantissant l'autonomie d'un administrateur au sein de n'importe quel environnement PowerShell :

Cmdlet	Rôle opérationnel	Exemple type
Get-Command	Identifier la commande adéquate selon le verbe d'action ou le nom de la ressource.	Get-Command -Verb Get -Noun *Disk*
Get-Help	Lire la documentation technique, la syntaxe des paramètres et les exemples d'implémentation.	Get-Help Set-Service -Examples

Get-Member	Révéler la structure interne de l'objet .NET en mémoire (Type, Propriétés et Méthodes).	`Get-Process
-------------------	---	--------------